

Finals PRACTICE Task 01

This SQL practice task focuses on cleaning and standardizing data within a `shipments_table` by formatting text strings with proper casing, uppercase letters, and trimmed whitespace.

It converts string representations of delivery and shipping dates into standard date formats to accurately calculate the transit time using the `datediff` function.

Conditional logic is applied via `CASE` statements to flag logical errors, such as delivery dates occurring before ship dates, and to correct negative weight entries by converting them to their absolute values.

The query employs the `row_number()` window function to partition the dataset, effectively identifying duplicate shipment records based on matching warehouse, destination, and shipping details.

Finally, a `DELETE` statement paired with a subquery is used to permanently remove those identified duplicate rows from the database, ensuring data integrity.

```
SELECT shipment_id,
proper_case(trim(origin_warehouse)) AS origin_warehouse,
proper_case(trim(destination_city)) AS odestination_city,
ucase(destination_state) AS destination_state,
proper_case(trim(carrier)) AS carrier,
str_to_date(delivery_date, '%y-%m-%d') AS delivery_date,
str_to_date(ship_date, '%y-%m-%d') AS ship_date,

datediff(delivery_date, ship_date) AS No_Of_Days_Delivered,

CASE
    WHEN (delivery_date) < (ship_date) THEN 'Invalid'
    WHEN (delivery_date) = (ship_date) THEN 'Same day delivery'
    ELSE 'Valid'

END as Delivery_status,

CASE
    WHEN weight_kg < 0 THEN abs(weight_kg)
    WHEN weight_kg = 0 THEN 0
    ELSE weight_kg
END as valid_weights,

row_number() over(
partition by origin_warehouse, destination_city, destination_state,
ship_date, carrier, CONVERT(weight_kg, char),
CONVERT(freight_cost, char)

ORDER BY shipment_id) as row_num
```

```

FROM shipment_db.shipments_table;

DELETE FROM shipments_table
WHERE shipment_id IN
(
select shipment_id FROM (
    SELECT shipment_id, row_number() over
    (
partition by origin_warehouse, destination_city, ship_date
ORDER BY shipment_id) as row_num
FROM shipments_table)
as sub
WHERE row_num > 1 );

```

The screenshot shows the MySQL Workbench interface. The main window displays a SQL query for Query 1, which is a DELETE statement using a subquery with a row_number() window function to delete duplicate shipment IDs based on origin_warehouse, destination_city, and ship_date. The query is as follows:

```

CASE
WHEN weight_kg < 0 THEN abs(weight_kg)
WHEN weight_kg = 0 THEN 0
ELSE weight_kg
END as valid_weights,
row_number() over(
partition by origin_warehouse, destination_city, destination_state,
ship_date, carrier, CONVERT(weight_kg, char),
CONVERT(freight_cost, char)

```

The Results grid shows the output of the query, displaying columns: shipment_id, origin_warehouse, destination_city, destination_state, carrier, delivery_date, ship_date, No_of_Days_Delivered, Delivery_status, valid_weights, and row_num. The results show 24 rows of data, with some rows marked as 'Invalid' due to negative weights or zero weights.

The bottom panel shows the 'Table: shipments_table' structure with columns: shipment_id (varchar(20)), origin_warehouse (text), destination_city (text), destination_state (text), carrier (text), ship_date (text), weight_kg (double), freight_cost (double), and shipment_status (text).

The bottom right panel shows the 'Action Output' window, which displays the execution of the DELETE statement, indicating that 24 rows were affected.